Existence is difference

Ren (𠈌)

Draft — April 2026

Abstract. An axiom system for existence. The symbol "=" resolves into three relations — native identity (≡), interaction (=), and convention (≈) — introduced through three orthogonal concerns: **Existence** (entities and their own inherent time), **Computable** (cost structure for real-world systems), and **ExternalTime** (an externally imposed time coordinate, parallel to Computable). Each commits to a minimal primitive vocabulary and a minimal set of axioms. The three relations, their separations, and the impossibility results that follow are derived within this structure. Mechanically verified in Coq.

1 Motivation

The symbol "=" appears in every line of mathematics but does not always name the same relation. Five statements using it, paired with the relation each actually asserts:

Statement	**Relation**	**Character**
$3 = 3$	$\equiv$	reflexive identity; no operator applied, nothing lost
$2 + 1 = 3$	$=$	value survives, provenance does not (3 is also $1 + 2$ or $0 + 3$)
$7 \bmod 3 = 1$	$=$	ten pre-images collapse to three residues; sound, not complete
$(a + b)^2 = a^2 + 2ab + b^2$	$=$	ring identity; both directions survive, many syntactic shapes for one polynomial
$\lim_{x \to 0} \sin(x)/x = 1$	$\approx$	no term equals 1; asserted as a limit, not computed

Five statements, one symbol, three distinct relations.

The three are pairwise disjoint: no pair of entities inhabits two of them at once. ≡ excludes = by distinctness (the interaction case requires $a \neq b$); ≡ excludes ≈ because a reflexive convention would demand $\text{interact}(a, c) \neq \text{interact}(a, c)$, which is absurd; and = excludes ≈ because convention is axiomatised to block interaction-kernel agreement at every viewpoint. Section Section 3 makes each exclusion explicit from the axioms and the formal definitions of the three relations.

The next section states the primitives and axioms of each layer, together with what the axioms jointly constrain and what they leave to the instance. A mechanical verification in Coq accompanies the text; the axioms themselves use only first-order logic with inductive types and are portable to any proof assistant supporting those.

Notation. Throughout, `code font` denotes a Coq identifier; math notation (=, ≡, ≈) denotes the paper-level relations of this text. `<>` is Leibniz inequality on terms (syntactic distinctness).

From Section Section 3 onward, the math = specifically refers to the paper-projection relation defined there.

2 Primitives and axioms

The framework separates its axioms along three orthogonal concerns:

- **Existence** — existence itself and the entity's own, inherent temporal axis. Every entity carries its own time in the form of viewpoints that move it; no external clock is assumed.
- **Computable** — cost structure for real-world systems. Interactions consume resources (capacity, storage, work); no step is free.
- **ExternalTime** — an externally imposed time coordinate, distinct from the entity's own temporal axis. Parallel to Computable, not downstream of it.

Each extension inherits Existence's primitives and theorems without modifying them.

2.1 Existence

One type, three primitives, five axioms.[1] `Entity` is the sole object type; `interact` is a binary operation that takes an entity and a viewpoint and returns what the entity looks like from that viewpoint. `convention_eq` is the one relation the framework itself names, constrained by a single axiom. The remaining two axioms state the framework's existence condition: without `existence` and `interact_with` there is nothing static to distinguish and nothing dynamic to observe.

Primitives.

Name	Type
`Entity`	`Type`
`interact`	`Entity -> Entity -> Entity`
`convention_eq`	`Entity -> Entity -> Prop`

Axioms.

Name	Statement
Interaction laws	
`interact_self`	`forall a, interact a a = a`

`interact_decidable`	`forall a b c,`
	`  {interact a c = interact b c} +`
	`  {interact a c <> interact b c}`

Existence	
`existence`	`exists a b, a <> b`
`interact_with`	`forall a, exists b, interact a b <> a`
Convention	
`convention_not_derivable`	`forall a b, convention_eq a b ->` `  forall c, interact a c <> interact b c`

Four notes on what the axioms jointly constrain.

[1] Formalised in `framework/Existence.v` (module type `ExistenceSig`, theory functor `ExistenceTheory`).

Self-identity. `interact_self` says an entity observed through itself is itself. This is the framework's only commitment about the self-viewpoint; every other pair of entities is treated symmetrically.

Existence and interact_with. `existence` and `interact_with` together supply what interaction itself requires: at least two distinct entities to relate, and at least one viewpoint in which something moves. Neither axiom alone suffices — with only `existence`, nothing happens; with only `interact_with`, there is nothing distinct to observe. Together they are the irreducible minimum for interaction to be meaningful at all. `interact_with` carries the entity's **own** temporal axis: time as a viewpoint inherent to the entity, not an external coordinate. No external clock is assumed or needed at the Existence layer.

Decidability. `interact_decidable` makes the kernel of interaction observable: at any common viewpoint, two entities' interactions either agree or disagree, and we can tell which constructively. This is what distinguishes $=$ (interaction, decidable at any viewpoint) from $\approx$ (convention, not reached by any interaction).

Convention. `convention_eq` is a relation an instance may assert; `convention_not_derivable` is the only framework commitment about it — whenever two entities are related by convention, some viewpoint must separate them under `interact`. Most instances set `convention_eq := False` (no conventions); the EpsilonDelta instance is the discriminating case, where `convention_eq` carries classical epsilon-delta convergence.

What the axioms leave free: the shape of `Entity`, the specific behaviour of `interact` at arbitrary pairs (`interact_decidable` only asks that we can tell agreement at the kernel, not what the output looks like), and whether `convention_eq` is inhabited. Those choices belong to the instance, not the framework.

2.2 Computable

Real-world systems pay for their interactions. Computable is the cost layer — three primitives, two axioms.[2] `info_size` is the capacity of an entity at a moment — the number of distinguishable states it carries. Accumulated over an interaction chain, cost splits into `storage_cost` (the per-step charge on the source's `info_size`) and `flip_cost` (one token per step, plus any growth in `info_size`). No interaction is free at this layer.

Primitives.

Name	Type
`info_size`	`Entity -> nat`
`storage_cost`	`Entity -> nat`
`flip_cost`	`Entity -> nat`

Axioms.

Name	Statement
Storage accumulation	
`storage_pays_capacity`	`forall a c,` `  interact a c <> a ->`

[2] Formalised in `framework/Computable.v` (module type `ComputableExistenceSig`, theory functor `ComputableExistenceTheory`).

	`storage_cost (interact a c)` `  = storage_cost a + info_size a`
Flip accumulation	

`flip_pays_work`	`forall a c,`
	`  interact a c <> a ->`
	`  flip_cost (interact a c)`
	`  = flip_cost a`
	`  + Nat.max 1 (info_size (interact a c)`
	`  - info_size a)`

Three notes on what each primitive records.

info_size. Finite by convention. Symbolic tags for classically uncountable sets appear as small finite values in instances, because the tag itself is one distinct state in a finite catalogue.

storage_cost. The accumulated charge on `info_size` across the interaction chain. Each non-identity interaction increases storage by the source's `info_size` — the price of carrying the source through the step. At the self-viewpoint, interaction is identity and storage is unchanged.

flip_cost. The accumulated operator count. Each non-identity interaction pays at least one token; if the interaction grows `info_size` by k, it pays k more. No interaction is free.

What the axioms leave free: the absolute values of `info_size`, `storage_cost`, and `flip_cost` at any specific entity; whether specific operators shrink, grow, or preserve `info_size`; and the identification of `info_size` with any particular physical unit. Those are instance commitments.

2.3 ExternalTime

ExternalTime introduces a time coordinate that is **not** the entity's own — an externally imposed counter, parallel to Computable (neither downstream of nor upstream from it), distinct from the internal temporal axis Existence already carries via `interact_with`. One primitive, one axiom.[3] An observer's coordinate that strictly advances on every non-identity interaction.

Primitives.

Name	Type
`external_time`	`Entity -> nat`

Axioms.

Name	Statement

`external_time_advances_on_nonself`	`forall a c,`
	`  interact a c <> a ->`
	`  external_time (interact a c)`
	`  > external_time a`

Two notes.

external_time. A strictly increasing counter. Unlike `info_size` (which an instance may shrink, grow, or hold fixed), `external_time` can only hold fixed on self-interactions — otherwise it strictly advances.

[3] Formalised in `framework/ExternalTime.v` (module type `ExternalTimeSig`, theory functor `ExternalTimeTheory`).

Why independent of Computable. Some instances carry a "value coordinate" that can stall under interaction — absorbing elements in a semilattice, for example, fix themselves. Such instances cannot satisfy `interact_with` through the value coordinate alone. `external_time` supplies a stall-free coordinate so that every entity has some viewpoint moving it, regardless of whether the value part moves.

What the axioms leave free: the initial value of `external_time`, how fast it advances, and any alignment with a physical notion of time. Those are instance commitments.

3 Three relations

Three relations on `Entity`.[4] The first two derive from `interact`, the framework's sole primitive operation on entities; the third is the one relation named at the signature level.

Definitions.

Symbol	Name	Definition
$\equiv$	native identity	`a = b` (Leibniz equality)
$=$	interaction	`a <> b` and `exists c, interact a c = interact b c`
$\approx$	convention	`convention_eq a b`

Each relation classifies by the kind of witness it admits. $\equiv$ is witness-free: reflexivity fixes it at every entity without any viewpoint needing to be produced. $=$ is single-witness: the instance must exhibit one explicit viewpoint c at which the two distinct entities' interactions agree. $\approx$ is witness-less: the instance commits to the equality at the signature level, and `convention_not_derivable` rules out interaction-kernel agreement at every viewpoint, leaving the equality asserted but structurally unreachable by any witness within the framework.

Trichotomy. The three relations are pairwise disjoint. No pair of entities can inhabit two of them at once.[5]

The three exclusions follow directly from the axioms:

- $\equiv$ excludes $=$: the $=$ definition requires `a <> b`, incompatible with the $\equiv$ requirement `a = b`.
- $\equiv$ excludes $\approx$: a reflexive convention would demand `interact a a <> interact a a` via `convention_not_derivable`, contradicting `interact_self`.
- $=$ excludes $\approx$: $\approx$ denies interaction-agreement at *every* viewpoint, while $=$ supplies such a viewpoint by definition.

Antipodal structure. $\equiv$ and $\approx$ sit at opposite ends of a single quantifier: both range over every viewpoint, but one declares universal agreement and the other universal disagreement.

$\equiv$	$\forall c, \text{interact}(a, c) = \text{interact}(b, c)$
$\approx$	$\forall c, \text{interact}(a, c) \neq \text{interact}(b, c)$

$=$ sits strictly between them as an existential: some viewpoint agrees, not necessarily all.

Immediate properties. From the axioms alone:

[4] Definitions `paper_equiv`, `paper_projection`, `paper_convention` in `results/Trichotomy.v` (functor `Make`). Derived results in this section come from `framework/Existence.v`'s functor `ExistenceTheory`.

[5] `paper_trichotomy_pairwise_disjoint` in `results/Trichotomy.v`.

- $\equiv$ is reflexive; $=$ is irreflexive (by `a <> b`); $\approx$ is irreflexive.[6]
- $\equiv$ is preserved at every viewpoint by Leibniz substitution: `a = b` implies `interact a c = interact b c` at every `c`.
- $\approx$ is destroyed by every interaction: `convention_eq a b` implies `interact a c <> interact b c` at every `c`, by `convention_not_derivable` applied directly.

The three relations therefore split the equality landscape by witness availability. $\equiv$ needs no witness; $=$ needs exactly one; $\approx$ admits none — its content is precisely that no witness exists within the framework. Classical math conflates these three into a single symbol and thereby loses the distinction between what is trivially the case, what is discharged by a single explicit witness, and what is asserted beyond the reach of any witness.

4 Derived consequences

The axioms of Section Section 2 produce a handful of statements that hold in every instance. None of them depend on what `Entity` is — only on the axioms that any instance must satisfy.

Terminal impossibility. No entity is fixed by every interaction. For any entity a, some viewpoint moves it.[7]

$$\forall a : \text{Entity}, \quad \exists b, \quad \text{interact}(a, b) \neq a$$

Direct from `interact_with`. The axiom rules out a terminal state where all viewpoints preserve the entity. In particular: no "perfectly stable" entity can exist under Existence alone. Any notion of stability or termination is an instance-level commitment beyond the base axioms, not a framework primitive.

Fixed point at self-viewpoint. Dually, every viewpoint fixes at least one entity — itself.[8]

$$\forall c : \text{Entity}, \quad \exists a, \quad \text{interact}(a, c) = a$$

The witness is $a \coloneqq c$, by `interact_self`. Combined with terminal impossibility, every viewpoint has a nonempty but proper fixed set — never all of `Entity`.

Dichotomy. At any common viewpoint, two entities agree or disagree, and the answer is decidable.[9]

$$\{\text{interact}(a, c) = \text{interact}(b, c)\} + \{\text{interact}(a, c) \neq \text{interact}(b, c)\}$$

This is what separates $=$ from $\approx$ at the verification level: $=$ demands a specific c at which agreement is decidable, $\approx$ declares disagreement at every c.

Convention forces distinctness. Two entities related by $\approx$ are distinct under every interaction, and in particular as entities.[10]

$$\text{convention\_eq}(a, b) \;\Rightarrow\; a \neq b$$

Applied at the self-viewpoint via `convention_not_derivable`, with $b \coloneqq a$: $\text{convention\_eq}(a, a)$ would imply $\text{interact}(a, a) \neq \text{interact}(a, a)$, contradicting `interact_self`. Hence $\approx$ is irreflexive — the specialisation of distinctness to the diagonal.

---

[6] `convention_eq_irreflexive` in `framework/Existence.v`.

[7] `is_terminal_impossible` in `framework/Existence.v`.

[8] `viewpoint_has_fixed_point`.

[9] `dichotomy`; follows from `interact_decidable`.

[10] `convention_eq_distinct`.

Observational equivalence. Two entities are *observationally equivalent* when every viewpoint produces the same result:

$$\mathrm{obs}(a, b) := \forall c, \quad \mathrm{interact}(a, c) = \mathrm{interact}(b, c)$$

This is the $\forall$-dual of $=$'s existential. obs and $\approx$ are structurally incompatible: obs asserts universal agreement, $\approx$ universal disagreement.[11]

$$\mathrm{obs}(a, b) \;\Rightarrow\; \neg\, \mathrm{convention\_eq}(a, b)$$

Applied at $c := a$: $\mathrm{obs}(a, b)$ gives $\mathrm{interact}(a, a) = \mathrm{interact}(b, a)$, i.e., $a = \mathrm{interact}(b, a)$ by `interact_self`; while $\mathrm{convention\_eq}(a, b)$ gives $\mathrm{interact}(a, a) \neq \mathrm{interact}(b, a)$, i.e., $a \neq \mathrm{interact}(b, a)$.

The pair $(\equiv, \approx)$ are the two antipodes; $(=, \mathrm{obs})$ are the existential and universal witnessings of interaction agreement. Their combinatorial table, using obs for observational equivalence:

	$\equiv$	$= (\exists c)$	obs $(\forall c)$	$\approx (\forall c, \neq)$
reflexive?	yes	no	yes	no
distinct?	no (Leibniz-equal)	yes	free	yes
derivable from interact?	trivially	yes	yes	no

Cost is strictly positive. At the Computable layer, every non-identity interaction pays a nonzero flip cost and never decreases storage cost. Formally:[12]

$$\mathrm{interact}(a, c) \neq a \quad \Rightarrow \quad \mathrm{storage}(a) \leq \mathrm{storage}(\mathrm{interact}(a, c)) \quad \wedge \quad \mathrm{flip}(a) < \mathrm{flip}(\mathrm{interact}(a, c))$$

By `flip_pays_work` (minimum flip per step) and `storage_pays_capacity` (storage charged by source info size). The strict inequality on flip is the Computable-layer statement that **distinguishing has positive minimum cost** — no non-identity step can be observed for free.

Combined with `existence` (at least two entities exist), every instance carries at least one achievable non-identity interaction, and that interaction pays at least one flip. The lower bound is tight and independent of instance details.

Why these costs are structural. The axioms are not chosen cosmetically — they follow from what "meaningful operation" requires. For a decision to carry information, the output must distinguish at least two outcomes (`existence`), each carrying at least one unit of `info_size`. Any operator producing those outcomes pays at least one flip token to transit between them (`flip_pays_work` minimum). These minima are not rounded up: they are the smallest values consistent with a non-trivial distinction occurring at all.

Abstraction cannot push any of the three below its minimum — a "finer" operator still pays at least one flip, a "smaller" entity still occupies at least one unit of capacity. Refinement in the other direction — splitting one operation into $m \geq 2$ sub-operations — only multiplies the count. Every sub-step still pays its flip; at k levels of refinement, total flip cost accumulates to at least m^k. The cost is not eliminated by making operations smaller; it redistributes multiplicatively.

Together: the minimum unit is 1 and refinement only stacks integers. No continuous reduction exists between adjacent cost values; every non-identity operation adds exactly as many flip tokens as it subdivides into. Computation is quantised by the same axioms that separate $=$ from $\approx$.

[11] `observational_equivalence_excludes_convention`.

[12] `both_costs_advance` in `framework/Computable.v`.

5 Instances

Three instances demonstrating that each relation is inhabited and non-trivial in at least one mathematically natural setting. Each is a module satisfying `ExistenceSig` (or an extension), verified in Coq.

5.1 Rationals: interaction equality via canonicalisation

`RationalRep` realises $=$ non-trivially. `Entity` is either a rational paired with a time stamp, or a canonicalising viewpoint:

```
REnt : Q -> nat -> Entity
CMark : nat -> Entity
```

Interaction at `CMark` reduces the rational via `Qred` (the stdlib reduction to lowest terms). Interaction with another `REnt` preserves the source rational and advances the time component.

`convention_eq := False`: rationals have no convention-level equalities — everything is resolvable at a viewpoint.

Writing $\underset{\mathbb{Q}}{\equiv}$ for Q-value equivalence (the setoid equality, e.g., $1/2 \underset{\mathbb{Q}}{\equiv} 2/4$), the central instance-level theorem states:[13]

$$(q_1 \neq q_2) \quad \wedge \quad \left(q_1 \underset{\mathbb{Q}}{\equiv} q_2\right) \quad \Rightarrow \quad \mathrm{REnt}(q_1, t) = \mathrm{REnt}(q_2, t)$$

The `CMark` viewpoint witnesses agreement after `Qred`. Executing `interact (REnt (1#2) 0) (CMark 0)` and `interact (REnt (2#4) 0) (CMark 0)` produces the same concrete term `REnt (1#2) 1`, verifying the equality computationally.

Numerical identities between specific rationals — e.g., $1/2 + 1/4 = 3/4$, or $3/6 = 1/2$ — all sit at $=$. The `CMark` viewpoint resolves each through `Qred`, collapsing distinct `Q` values with the same reduced form into the same canonical representation.

5.2 Cauchy sequences: convention equality via epsilon-delta

`CauchyReal` realises $\approx$ non-trivially. `Entity` is either a Cauchy term (a structural sequence schema), or an evaluation viewpoint:

```
REnt : CauchyTerm -> nat -> Entity
CEval : nat -> nat -> Entity
```

`CauchyTerm` is an inductive grammar (`CTConst`, `CTInvSucc`, `CTSum`, `CTNeg`, `CTScale`, `CTMul`) denoting sequences $\mathbb{N} \to \mathbb{Q}$ via a `denote` function. Two terms are *cauchy-equivalent* when their denotations converge together in the ε-δ sense:

$$\texttt{cauchy\_equivalent s1 s2} := \forall k, \exists N, \forall n \geq N, \texttt{Qabs (denote s1 n - denote s2 n)} \leq 1/(k+1)$$

The relation *pointwise_distinct* asks that the denoted sequences disagree at every index.

convention_eq on `CauchyReal` is *cauchy_equivalent AND syntactically distinct AND pointwise_distinct*. The first is the classical limit condition; the second ensures the $\approx$ predicate is meaningful (non-reflexive); the third is what `convention_not_derivable` requires.[14]

A central instance witness: the constant-1 sequence and the sequence $1 + 1/(n+1)$ are $\approx$.[15]

---

[13] `rational_equivalent_paper_projection` in `existences/RationalRep.v`.

[14] `cauchy_pointwise_distinct_convention` in `existences/CauchyReal.v`.

[15] `const_sum_convention_eq` in `tests/CauchyRealTest.v`. The ε-δ inequality is proved by taking $N := k$.

```
REnt (CTConst 1) 0 ≈ REnt (CTSum (CTConst 1) CTInvSucc) 0
```

Both denote sequences converging to 1, differ at every finite index by $1/(n+1)$. No viewpoint in `CauchyReal`'s signature bridges them — any evaluation at CEval(n) produces `CTConst 1` on the left and `CTConst (1 + 1/(n + 1))` on the right, concrete and distinct.

$=$ also appears in `CauchyReal`, alongside $\approx$: syntactically distinct but *pointwise-equal* terms collapse at a single `CEval` viewpoint. Polynomial identities on `CauchyTerm` are the headline examples: double negation $-(-x) = x$, cancellation $x + (-x) = 0$, associativity of `CTMul`, and the binomial expansion

$$(a+b)^2 = a^2 + 2ab + b^2$$

are each pointwise equalities between syntactically distinct terms — the ring tactic closes them at the `denote` level at every n.[16] Classical algebra is exactly the $=$ layer at the term grammar.

Quadratic decay illustrates both:

```
CTMul CTInvSucc CTInvSucc ≈ CTConst 0
```

Sequence $1/(n+1)^2$ converges to 0, pointwise-distinct at every n.[17]

5.3 Lattices: only identity

`LatticeComputable` (and related `SemilatticeInstances`)[18] realise an instance at the other end: only $\equiv$ is non-trivially inhabited. `Entity` is a lattice value paired with a time counter; `interact` reduces to the idempotent binary operation componentwise; `convention_eq := False`.

Here $=$ reduces to $\equiv$ up to the time component — no finite witness makes two distinct lattice values agree under interaction other than the degenerate self-pair. `LatticeComputable` serves as the baseline instance where every paper-level equality is native.

5.4 Summary

Three instances give the three regions:

Instance	$\equiv$	$=$	$\approx$
`LatticeComputable`	non-trivial	trivial	none
`RationalRep`	non-trivial	**non-trivial**	none
`CauchyReal`	non-trivial	non-trivial	**non-trivial**

The three-layer framework is not empty at any layer. Each relation is populated in at least one mathematically recognisable setting, and each instance's $\approx$ statement mirrors a standard classical identity whose status the framework now makes precise.

6 Cross-system coherence

A framework that distinguishes relations must survive translation between instances: the three relations should carry from one axiomatic system to another along interaction-preserving maps. `ExistenceMorphism` supplies the machinery; the `RationalRep -> CauchyReal` map supplies the concrete test.

---

[16] `double_neg_pointwise`, `sum_neg_pointwise`, `mul_assoc_pointwise`, `binom_square_pointwise` in `results/CauchyLimits.v`.

[17] `invsucc_squared_convention_eq_zero`.

[18] `existences/LatticeModel.v`, `existences/SemilatticeInstances.v`, `framework/LatticeExternalTimed.v`.

Morphism. A function $\varphi : D_1.\text{Entity} \to D_2.\text{Entity}$ between two `ExistenceSig` instances *preserves interaction* when

$$\varphi(\text{interact}_1(a, b)) \;=\; \text{interact}_2(\varphi(a), \varphi(b))$$

holds for every pair. A morphism is *injective* when distinct source entities map to distinct target entities.

Such a morphism automatically preserves the self-viewpoint: at (a, a) the preservation equation combined with `interact_self` yields $\varphi(a) = \text{interact}_2(\varphi(a), \varphi(a))$. It also carries agreement: if a and b interact to the same result at some source viewpoint c, their images do so at $\varphi(c)$.[19]

The constant-sequence embedding. Let φ send a rational representation to the corresponding constant Cauchy sequence:

$$\begin{gathered}\text{REnt}_{\text{R}(q,t)} \mapsto \text{REnt}_{\text{C}(\text{CTConst}(q),t)} \\ \text{CMark}(t) \mapsto \text{CEval}(0, t)\end{gathered}$$

φ is interaction-preserving and injective.[20] Both properties are direct case analysis on the source entity: `REnt` images preserve under source-preserving interaction and under `CEval` viewpoints (where `denote (CTConst q) n = q` makes the `CEval` reduction match `CMark`'s `Qred`); injectivity is the combination of Leibniz equality on rationals and constructor disjointness.

Transfer of interaction equality. The rational fact "1/2 and 2/4 are = in `RationalRep`" lifts to "`CTConst (1/2)` and `CTConst (2/4)` are = in `CauchyReal`", purely by framework machinery.[21]

Given the source witness (the `CMark` viewpoint), `morphism_carries_agreement` provides $\text{CEval}(0, 0)$ — which is $\varphi(\text{CMark}(0))$ — as the target-side witness; injectivity preserves the distinctness clause. No `Qred_complete` invocation in the target, no fresh ε-δ: the equality transfers in one step through the morphism.

Image shape and strict non-liftability. φ's image is the `REnt (CTConst q) t` shapes (any rational q, any time t) together with the `CEval 0 t` shapes.[22] Any CauchyReal entity outside this image — `CTSum` shapes, `CTInvSucc`, `CTMul` products — has no φ pre-image. In particular, the $\approx$ pair from Section Section 5.2 (the constant-1 sequence paired with $1 + 1/(n + 1)$) lies outside φ's range: the second component is a `CTSum` shape.

The stronger statement — no φ-image pair can be $\approx$ at all — is also true.[23]

$\approx$-equality in `CauchyReal` is strictly richer than $=$-equality in `RationalRep` carried through φ: it exposes structure in the Cauchy system that the rational system cannot describe. The framework's equality layering is therefore not preserved under arbitrary morphism — specifically, it may *grow* downstream, and the inability to lift $\approx$ is the framework-level statement of that asymmetry.

Factorisation through product, pullback, and pushout. Beyond morphism alone, the framework carries three further categorical constructions that organise cross-system structure: the *product* $\text{RR} \times \text{CR}$ (entities are joint pairs), the *pullback* $\{(r, c) : \varphi(r) = c\}$ (the graph of φ sitting inside the product), and the *pushout* gluing `RationalRep` to `CauchyReal` along the span $\text{RR} \xleftarrow{\text{id}} \text{RR} \xrightarrow{\varphi} \text{CR}$. Each has a Coq functor whose axioms are discharged from the underlying instances.

---

[19] `morphism_carries_agreement` in `framework/ExistenceMorphism.v`.

[20] `phi_preserves_interact`, `phi_injective` in `results/RationalToCauchyMorphism.v`.

[21] `halves_paper_projection_in_cauchyreal`.

[22] `phi_image_shape` in `results/RationalToCauchyMorphism.v`.

[23] `phi_cannot_witness_convention`; the proof passes through a Q-archimedean bound showing that ε-δ convergence between two constant sequences forces their rational values equal, which contradicts `pointwise_distinct`.

These constructions compose into a single factoring of φ:[24]

$$\varphi = \rho \circ \mathrm{inj}_1 \circ \mathrm{fst} \circ \mathrm{diag}_\varphi$$

where $\mathrm{diag}_\varphi(r) = (r, \varphi(r))$ places an entity on the graph (a pullback element), fst projects it back to `RationalRep`, inj_1 lifts into the pushout class, and ρ is the universal arrow from the cocone $(\mathrm{CR}, \varphi, \mathrm{id})$ back to `CauchyReal`. Every arrow is a framework-level construction; no ad-hoc step appears anywhere in the chain.

The paper's machinery therefore closes on itself: the morphism responsible for carrying $=$ between instances is itself a composition of framework primitives.

7 Conclusion

The single symbol $=$ in classical mathematics carries three distinct relations:

- $\equiv$, tautological identity, verified by reflexivity alone;
- $=$, interaction agreement, verified at a specific viewpoint;
- $\approx$, convention, asserted and provably unreachable by any interaction.

The framework separates these by committing to a minimal vocabulary (one type, three primitive operations, five axioms at the base layer) and deriving the trichotomy structure and its pairwise disjointness from the axioms alone. Classical identities assemble along the three relations as follows:

Classical identity	**Framework relation**
$3 = 3$	$\equiv$
$1/2 = 2/4$	$=$ (via `Qred`)
$(a+b)^2 = a^2 + 2ab + b^2$	$=$ (via pointwise denotation)
$\lim_{n\to\infty} 1 + 1/n = 1$	$\approx$ (via ε-δ)
$1/n^2 \to 0$	$\approx$ (quadratic decay)

Three instances — `RationalRep`, `CauchyReal`, and `LatticeComputable` — together populate all three relations and demonstrate that each is non-trivial in at least one mathematically natural setting. A morphism between the first two carries $=$ forward intact and leaves $\approx$ strictly behind, showing both that the framework is coherent across systems and that the relations are not a uniform relabelling — the $\approx$ layer expresses content the lower-layer system cannot.

Verification. Every statement — framework axioms, derived consequences, and all instance theorems cited — is mechanically verified in Coq, with zero `Admitted` and no external postulates. A single meta-axiom asserts the existence of quotient types and is used only by `ExistencePushout.v`; no result in this paper invokes it. The repository contains the primitives, axioms, instance constructions, and the cross-system morphism in machine-checked form.

The framework says, in one line: *mathematics has been working with three equalities and spelling them the same way.*

[24] `phi_factors_via_pullback_pushout` in `results/RationalCauchyFactorization.v`.

8 References

The paper does not invoke specific prior results, but the framework and its mechanical verification draw on the following bodies of work:

- The Coq Development Team. *The Coq Proof Assistant Reference Manual*. INRIA, continually updated. https://coq.inria.fr. The mechanical verification lives in Coq; the Rocq fork shares the same logical core and is interoperable.
- H. B. Curry and R. Feys. *Combinatory Logic*, Volume 1. North-Holland, 1958. The Curry–Howard correspondence — the reading of proofs as programs — is the substrate on which mechanical verification is meaningful: a Coq term of type P is, under this correspondence, a proof of P. The paper's "$= / \approx$" separation is intelligible in a setting where "a witness is a program" is the default stance.
- C. E. Shannon. "A Mathematical Theory of Communication". *Bell System Technical Journal* 27 (1948). Referenced for information-theoretic framing of distinguishability and capacity.
- A. N. Kolmogorov. "Three approaches to the quantitative definition of information". *Problems of Information Transmission* 1(1),
  1965. Referenced for the minimum-cost intuition

  underlying the quantization argument.